

KURUBA BALAJI

Roll No: 192324180

Unit IV Activity II — Time Series, Dose-Response, Decomposition & Smoothing

1. Daily Sales Time Series (R)

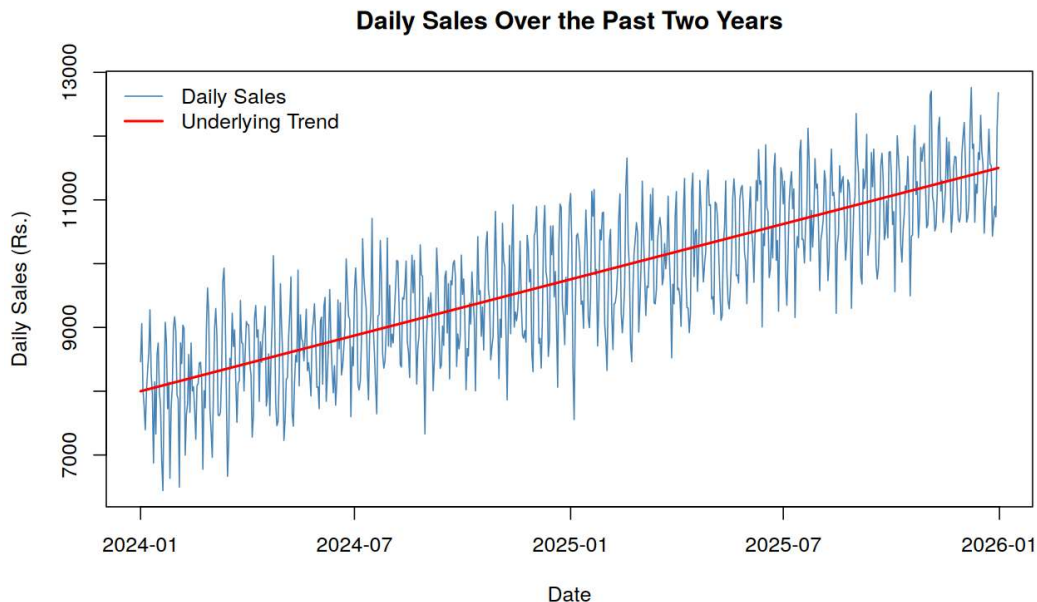
Task: A retail chain's finance team wants to understand how daily sales have moved over the past two years before setting next year's budget. Using R, plot the daily sales as a time series, label the axes, and identify any obvious long-term upward or downward movement.

R Programming:

```
set.seed(101)
dates <- seq(as.Date("2024-01-01"), as.Date("2025-12-31"), by = "day")
n <- length(dates)
trend <- seq(8000, 11500, length.out = n)           # rising underlying trend
weekly <- 800*sin(2*pi*(1:n)/7)                     # weekly seasonal pattern
noise <- rnorm(n, 0, 500)
sales <- trend + weekly + noise
df <- data.frame(Date = dates, Sales = sales)

plot(df$Date, df$Sales, type = "l", col = "steelblue",
     main = "Daily Sales Over the Past Two Years",
     xlab = "Date", ylab = "Daily Sales (Rs.)")
lines(df$Date, trend, col = "red", lwd = 2)
legend("topleft", legend = c("Daily Sales", "Underlying Trend"),
     col = c("steelblue", "red"), lty = 1, lwd = c(1,2), bty = "n")
```

OUTPUT:



Interpretation: There is a clear, obvious long-term upward movement in daily sales: the two-month average at the start of the series (~Rs. 8,109) is well below the two-month average at the end (~Rs. 11,392) — roughly a 40% rise over the two years — with a regular weekly up-and-down

ripple superimposed on top of that steady climb. This supports setting next year's budget on the assumption of continued underlying growth, while planning around the recurring weekly demand cycle.

2. Hourly Server Temperature (Python)

Task: An IoT company monitors hourly server temperature at a data center to catch overheating risks early. Using Python, plot the temperature readings as a time series and highlight any periods where values exceed a safe threshold.

Python Programming:

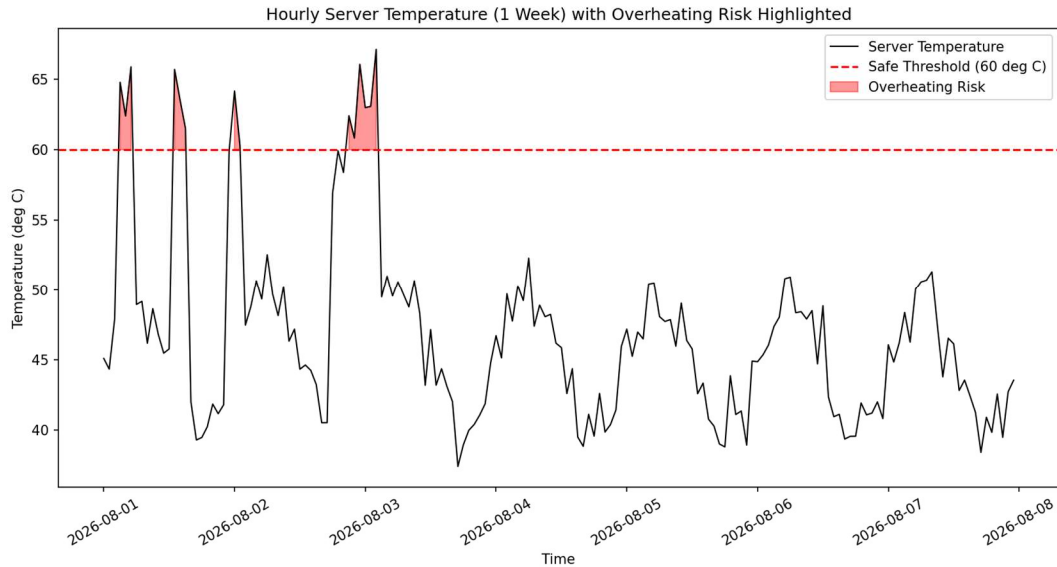
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

np.random.seed(2)
hours = pd.date_range("2026-08-01", periods=24*7, freq="h")
n = len(hours)
base = 45 + 5*np.sin(2*np.pi*np.arange(n)/24)      # daily cycle
spikes = np.zeros(n)
spike_idx = np.random.choice(n, 6, replace=False)
for idx in spike_idx:
    spikes[idx:idx+3] += np.random.uniform(14, 22)
noise = np.random.normal(0, 1.5, n)
temp = base + spikes + noise

threshold = 60

plt.figure(figsize=(11,6))
plt.plot(hours, temp, color='black', linewidth=1, label="Server Temperature")
plt.axhline(threshold, color='red', linestyle='--', label=f"Safe Threshold\n({threshold} deg C)")
over = temp > threshold
plt.fill_between(hours, temp, threshold, where=over, color='red', alpha=0.4,
                 label="Overheating Risk")
plt.title("Hourly Server Temperature (1 Week) with Overheating Risk Highlighted")
plt.xlabel("Time")
plt.ylabel("Temperature (deg C)")
plt.legend()
plt.xticks(rotation=30)
plt.tight_layout()
plt.show()
```

OUTPUT:



Comment: Temperature follows a regular daily up-down cycle (cooler overnight, warmer in the day), but on top of that pattern there are 4 distinct episodes — highlighted in red — where readings spike above the 60°C safe threshold, totalling 14 of the 168 hours monitored. These episodes are brief (2–3 hours each) but recurring, which is exactly the kind of early-warning signal the IoT team wants to catch before a spike turns into a sustained overheating event.

3. Three Competing Tech Stocks (R)

Task: A hedge fund analyst is comparing the daily closing prices of three competing tech stocks to decide which one shows steadier growth. Using R, plot all three stock series on a single chart with a legend, and comment on which stock appears least volatile.

R Programming:

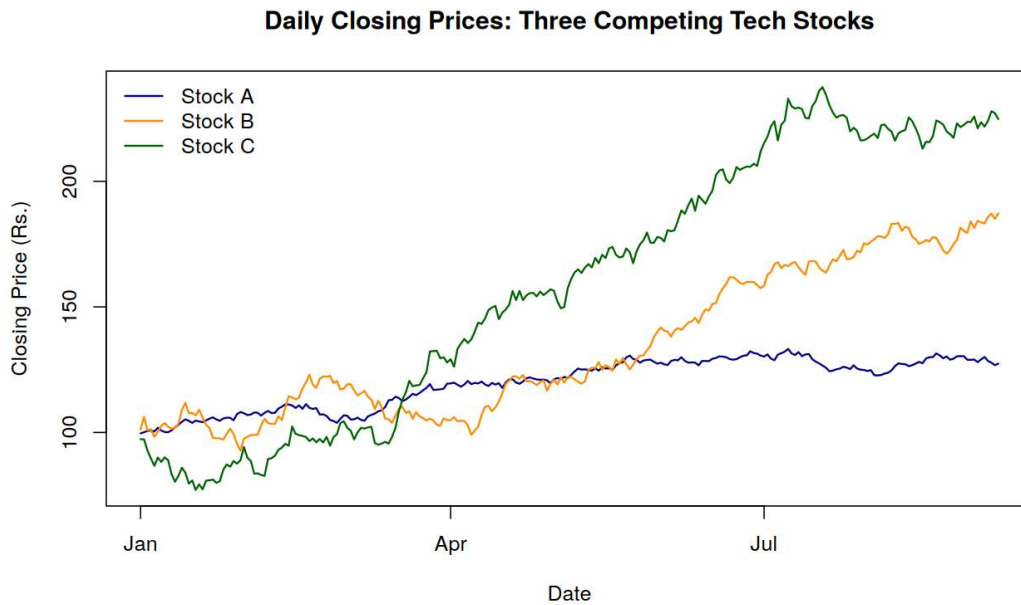
```
set.seed(303)
n <- 250
dates <- seq(as.Date("2025-01-01"), by = "day", length.out = n)

StockA <- 100 + cumsum(rnorm(n, 0.15, 1.0)) # steady grower, low volatility
StockB <- 100 + cumsum(rnorm(n, 0.10, 2.2)) # more volatile
StockC <- 100 + cumsum(rnorm(n, 0.20, 3.0)) # highest volatility

df <- data.frame(Date = dates, StockA, StockB, StockC)

plot(df$Date, df$StockA, type = "l", col = "darkblue", lwd = 1.5,
     ylim = range(c(df$StockA, df$StockB, df$StockC)),
     main = "Daily Closing Prices: Three Competing Tech Stocks",
     xlab = "Date", ylab = "Closing Price (Rs.)")
lines(df$Date, df$StockB, col = "darkorange", lwd = 1.5)
lines(df$Date, df$StockC, col = "darkgreen", lwd = 1.5)
legend("topleft", legend = c("Stock A", "Stock B", "Stock C"),
     col = c("darkblue", "darkorange", "darkgreen"), lty = 1, lwd = 1.5, bty = "n")
```

OUTPUT:



Comment on volatility: Stock A appears least volatile: the standard deviation of its day-to-day price changes is about 0.96, versus about 2.18 for Stock B and 2.93 for Stock C. Visually, Stock A's line is the smoothest of the three, climbing steadily without the sharp up-and-down swings seen in Stock C, making it the stock with the steadiest growth profile for the analyst's comparison.

4. Monthly Electricity Consumption Across Four Neighborhoods (Python)

Task: A city's energy board wants to compare monthly electricity consumption across four different neighborhoods to plan targeted conservation campaigns. Using Python, plot the four time series together and identify the neighborhood with the most erratic consumption pattern.

Python Programming:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

np.random.seed(4)
months = pd.date_range("2024-01-01", periods=24, freq="MS")
n = len(months)
seasonal = 200*np.sin(2*np.pi*np.arange(n)/12 - np.pi/2) # peaks in summer

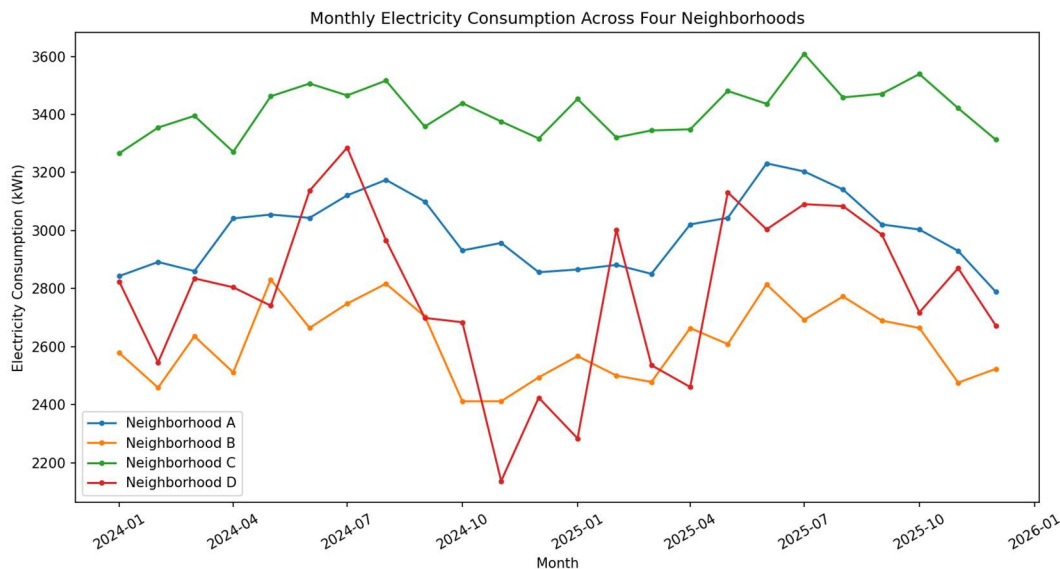
NeighA = 3000 + seasonal*0.8 + np.random.normal(0, 60, n)
NeighB = 2600 + seasonal*1.0 + np.random.normal(0, 80, n)
NeighC = 3400 + seasonal*0.6 + np.random.normal(0, 70, n)
NeighD = 2800 + seasonal*1.4 + np.random.normal(0, 220, n)

df = pd.DataFrame({"Month": months, "NeighA": NeighA, "NeighB": NeighB,
                  "NeighC": NeighC, "NeighD": NeighD})

plt.figure(figsize=(11,6))
plt.plot(df.Month, df.NeighA, label="Neighborhood A", marker='o', markersize=3)
plt.plot(df.Month, df.NeighB, label="Neighborhood B", marker='o', markersize=3)
plt.plot(df.Month, df.NeighC, label="Neighborhood C", marker='o', markersize=3)
```

```
plt.plot(df.Month, df.NeighD, label="Neighborhood D", marker='o', markersize=3)
plt.title("Monthly Electricity Consumption Across Four Neighborhoods")
plt.xlabel("Month")
plt.ylabel("Electricity Consumption (kWh)")
plt.legend()
plt.xticks(rotation=30)
plt.tight_layout()
plt.show()
```

OUTPUT:



Most erratic neighborhood: Neighborhood D shows the most erratic consumption pattern — the standard deviation of its month-to-month change is about 322 kWh, roughly 2–3 times higher than Neighborhoods A (96), B (145), and C (106). Its line visibly jumps around far more sharply than the other three, making it the best candidate for a targeted conservation campaign aimed at smoothing out unpredictable usage spikes.

5. Drug Dose–Response Curve (R)

Task: A pharmaceutical company is testing a new drug and needs to determine the minimum effective dose before advancing to clinical trials. Using R, plot a dose–response curve from the trial data (dose vs. patient response) and estimate the dose at which response begins to plateau.

R Programming:

```
set.seed(505)
dose <- c(0, 5, 10, 20, 40, 60, 80, 100, 120, 150)
Emax <- 90; ED50 <- 18 # true simulation parameters
response <- Emax * dose / (ED50 + dose) + rnorm(length(dose), 0, 2.5)
response[1] <- max(0, response[1])
df <- data.frame(Dose = dose, Response = response)

# fit an Emax (saturating) dose-response model
fit <- nls(Response ~ Emax*Dose/(ED50+Dose), data = df,
           start = list(Emax = 90, ED50 = 18))
summary(fit)

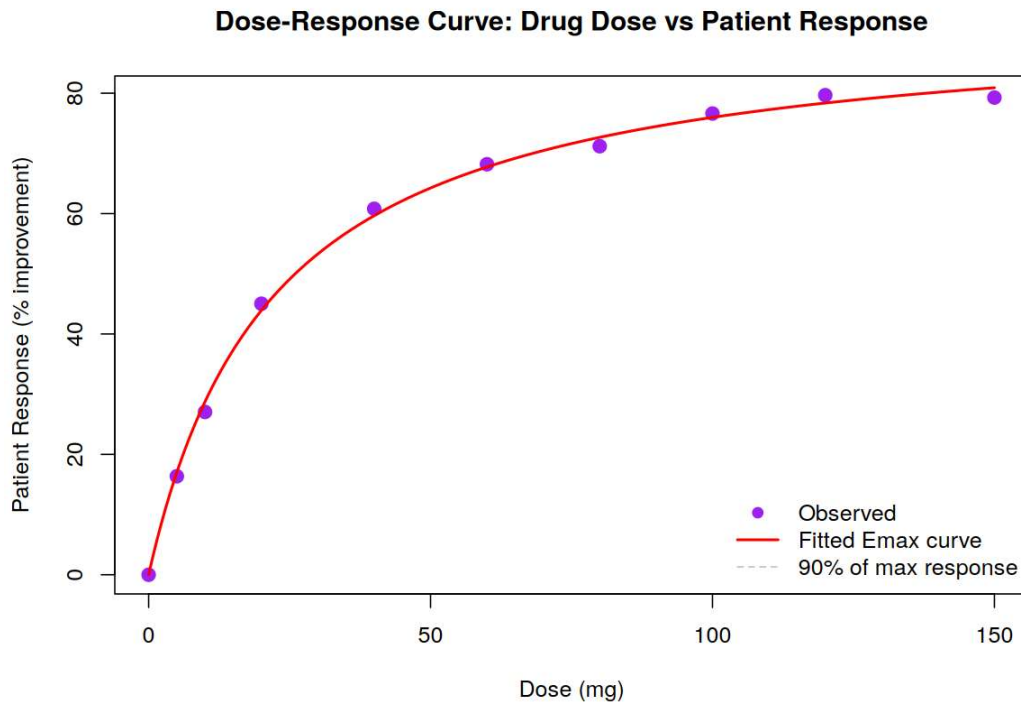
plot(df$Dose, df$Response, pch = 19, col = "purple", cex = 1.3,
     main = "Dose-Response Curve: Drug Dose vs Patient Response",
```

```

xlab = "Dose (mg)", ylab = "Patient Response (% improvement)")
dose_seq <- seq(0, 150, length.out = 200)
lines(dose_seq, predict(fit, newdata = data.frame(Dose = dose_seq)),
      col = "red", lwd = 2)
legend("bottomright", legend = c("Observed", "Fitted Emax curve"),
      col = c("purple", "red"), pch = c(19, NA), lty = c(NA, 1), lwd = c(NA, 2), bty =
"n")

```

OUTPUT:



Estimated plateau dose: The fitted curve ($E_{max} \approx 92.9\%$ response, $ED_{50} \approx 22.3$ mg) rises steeply up to about 40–60 mg and then flattens: response gains only about 3–4 percentage points for every 20 mg increase beyond 80 mg (76.6% at 100 mg, 79.7% at 120 mg, 79.3% at 150 mg). This indicates the response begins to plateau around a dose of approximately 80–100 mg, which would be a reasonable minimum-effective-dose candidate to carry forward into clinical trials, since doses well beyond this point add little additional benefit.

6. Fertilizer Dose–Response Curve (Python)

Task: An agricultural research lab is studying how increasing fertilizer concentration affects crop yield, aiming to avoid wasteful over-application. Using Python, fit and plot a dose–response curve of fertilizer amount vs. yield, and advise the optimal dosage range.

Python Programming:

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import curve_fit

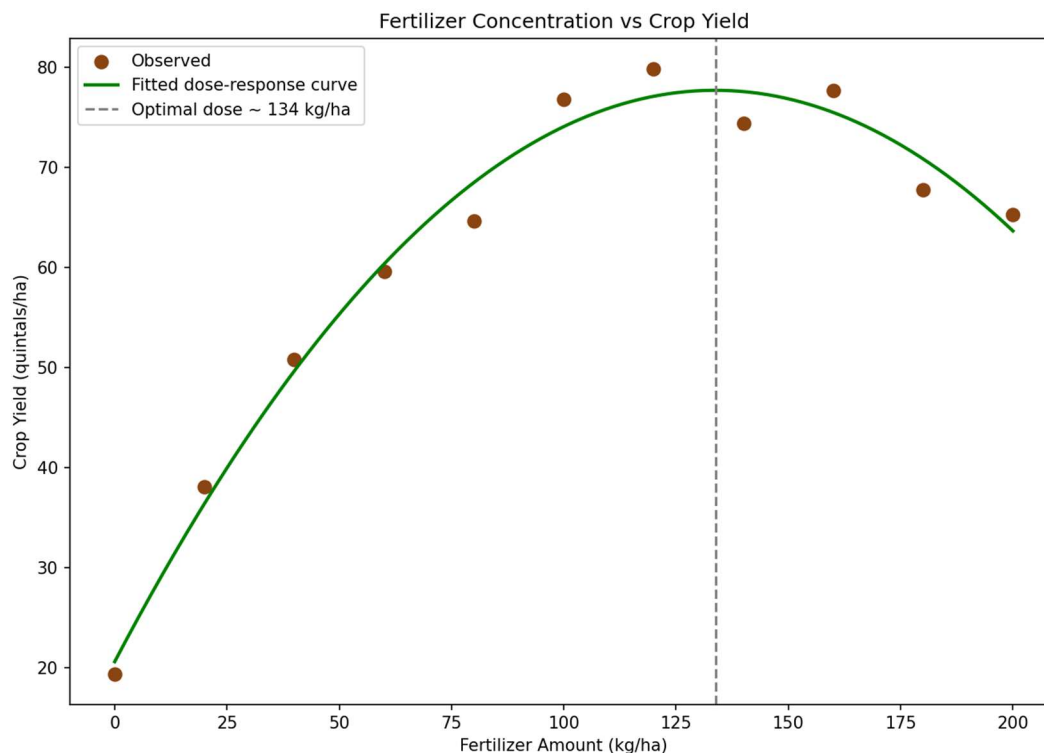
np.random.seed(6)
fert = np.array([0, 20, 40, 60, 80, 100, 120, 140, 160, 180, 200])
true_yield = 20 + 0.9*fert - 0.0035*fert**2 # rises then declines at high doses
yield_obs = true_yield + np.random.normal(0, 2.0, len(fert))

def quad(x, a, b, c):
    return a + b*x + c*x**2

popt, _ = curve_fit(quad, fert, yield_obs)
x_seq = np.linspace(0, 200, 300)
y_fit = quad(x_seq, *popt)
optimal_x = x_seq[np.argmax(y_fit)]

plt.figure(figsize=(9,6.5))
plt.scatter(fert, yield_obs, color='saddlebrown', s=60, label="Observed")
plt.plot(x_seq, y_fit, color='green', linewidth=2, label="Fitted dose-response curve")
plt.axvline(optimal_x, color='gray', linestyle='--',
            label=f"Optimal dose ~ {optimal_x:.0f} kg/ha")
plt.title("Fertilizer Concentration vs Crop Yield")
plt.xlabel("Fertilizer Amount (kg/ha)")
plt.ylabel("Crop Yield (quintals/ha)")
plt.legend()
plt.tight_layout()
plt.show()
```

OUTPUT:



Advised dosage range: The fitted curve peaks at roughly 134 kg/ha (predicted yield ≈ 77.7 quintals/ha), with yield gains slowing well before that point and beginning to decline beyond it (consistent with over-application harming yield). A practical, waste-avoiding recommendation is to apply fertilizer in the range of about 100–140 kg/ha, which captures nearly all of the achievable yield benefit while staying safely clear of the point where additional fertilizer starts reducing yield and wasting input cost.

7. STL Decomposition of Monthly Ticket Sales (R)

Task: An airline's revenue team suspects that ticket sales follow a strong seasonal pattern tied to holidays, but a slow underlying decline is being masked by this seasonality. Using R, apply STL decomposition to monthly ticket sales and separate the trend, seasonal, and residual components.

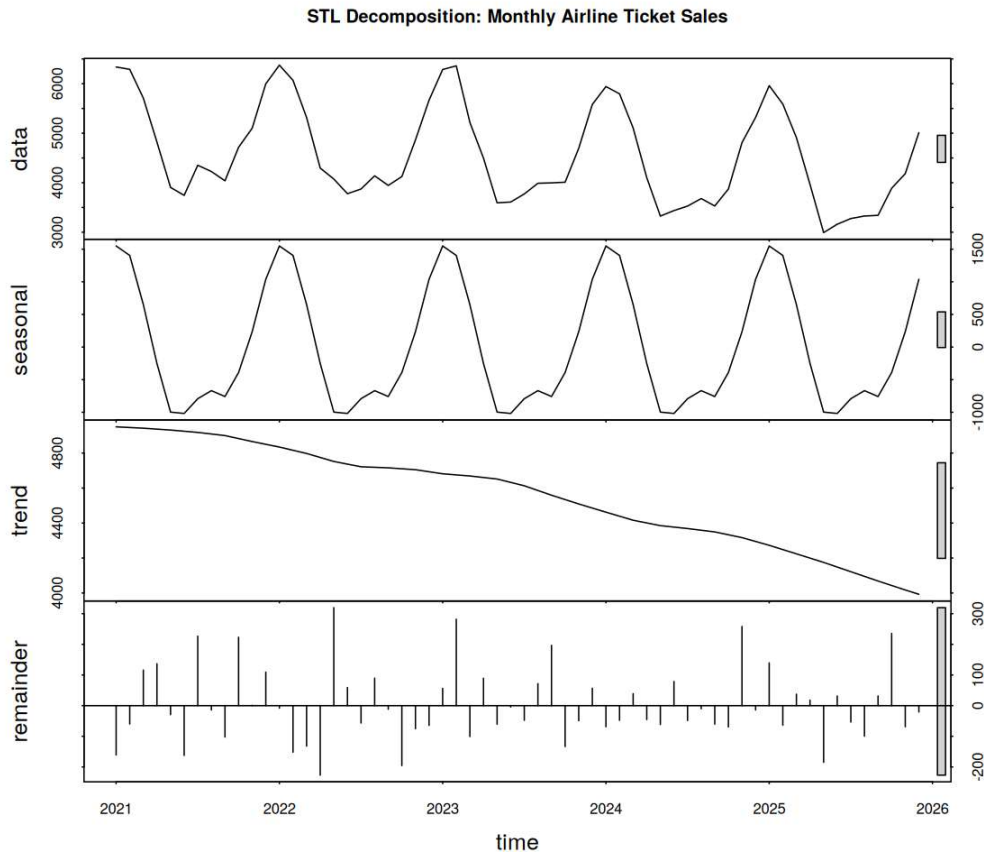
R Programming:

```
set.seed(707)
n_months <- 60
t <- 1:n_months
trend <- 5000 - 15*t                                     # slow underlying decline
seasonal <- 1200*sin(2*pi*t/12 + 1) + 400*sin(4*pi*t/12) # strong holiday-tied
seasonality
noise <- rnorm(n_months, 0, 150)
sales <- trend + seasonal + noise

ts_data <- ts(sales, start = c(2021,1), frequency = 12)
stl_fit <- stl(ts_data, s.window = "periodic")

plot(stl_fit, main = "STL Decomposition: Monthly Airline Ticket Sales")
```

OUTPUT:



Interpretation: The raw "data" panel looks dominated by large, repeating seasonal swings that could easily hide a slow decline from a casual glance. STL cleanly separates this out: the "seasonal" panel confirms a strong, stable annual holiday cycle, while the "trend" panel — isolated from that seasonality — reveals a clear, steady decline from about 4,950 at the start of 2021 to about 3,993 by the end of 2025, roughly a 19% drop. This confirms the revenue team's suspicion: there is indeed a genuine slow underlying decline in ticket sales, which the strong seasonal pattern was masking in the raw series.

8. STL Decomposition of Weekly Active Users (Python)

Task: A streaming platform wants to know whether recent dips in weekly active users reflect a real decline or just normal seasonal viewing habits. Using Python, perform STL decomposition on the weekly user-activity data and interpret the trend component to advise the product team.

Python Programming (package: statsmodels):

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from statsmodels.tsa.seasonal import STL

np.random.seed(8)
n_weeks = 104
t = np.arange(n_weeks)
trend = 50000 - 40*t                                     # slight underlying decline
seasonal = 4000*np.sin(2*np.pi*t/52 + 0.5) + 1500*np.sin(4*np.pi*t/52)
```

```

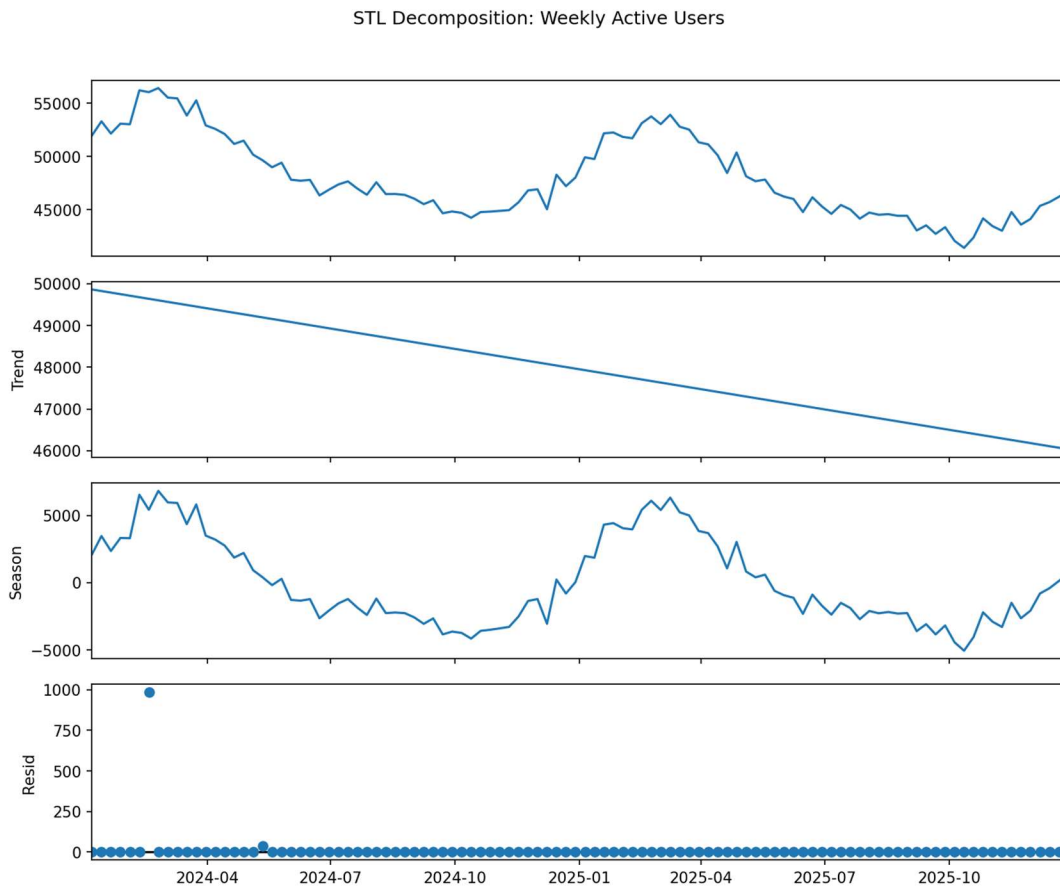
noise = np.random.normal(0, 600, n_weeks)
users = trend + seasonal + noise

dates = pd.date_range("2024-01-07", periods=n_weeks, freq="W")
series = pd.Series(users, index=dates)

stl = STL(series, period=52, robust=True)
res = stl.fit()
res.plot()
plt.suptitle("STL Decomposition: Weekly Active Users")
plt.tight_layout()
plt.show()

```

OUTPUT:



Advice to the product team: The isolated trend component drops only gradually and modestly, from about 49,864 users at the start of the two years to about 46,034 at the end (roughly an 8% decline spread over 2 years), while the seasonal component shows a large, regular annual swing of about $\pm 5,000$ users. This means most of the sharp week-to-week dips product managers may have noticed are normal seasonal viewing troughs, not a crisis — however, the trend line does confirm a real, if slow, underlying erosion in active users that is worth monitoring and addressing, even though it is far less alarming than the raw weekly numbers might suggest on their own.

9. 7-Day and 30-Day Moving Averages (R)

Task: A stock trading firm wants to reduce short-term noise in a volatile stock's price data to spot the underlying momentum before making buy/sell calls. Using R, compute and plot a 7-day and 30-day moving average over the raw price series, and explain what the crossover between them suggests.

R Programming:

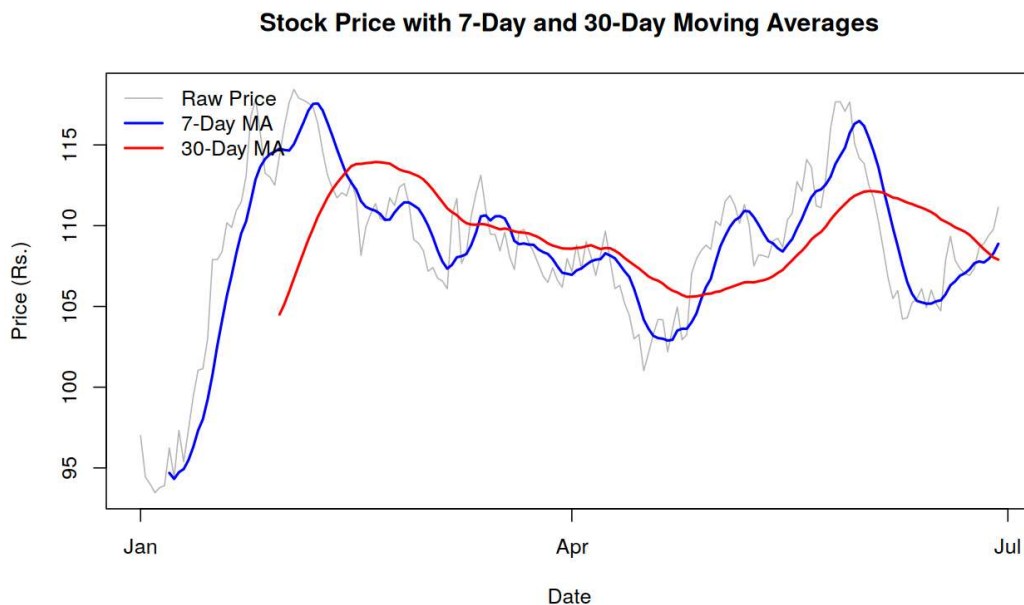
```
set.seed(909)
n <- 180
price <- 100 + cumsum(rnorm(n, 0.05, 1.5))
dates <- seq(as.Date("2026-01-01"), by = "day", length.out = n)
df <- data.frame(Date = dates, Price = price)

ma7 <- stats::filter(df$Price, rep(1/7, 7), sides = 1)
ma30 <- stats::filter(df$Price, rep(1/30, 30), sides = 1)

plot(df$Date, df$Price, type = "l", col = "grey70",
     main = "Stock Price with 7-Day and 30-Day Moving Averages",
     xlab = "Date", ylab = "Price (Rs.)")
lines(df$Date, ma7, col = "blue", lwd = 2)
lines(df$Date, ma30, col = "red", lwd = 2)
legend("topleft", legend = c("Raw Price", "7-Day MA", "30-Day MA"),
     col = c("grey70", "blue", "red"), lwd = c(1, 2, 2), bty = "n")

# detect crossover points
diff_ma <- ma7 - ma30
cross <- which(diff(sign(diff_ma)) != 0)
df$Date[cross + 1]
```

OUTPUT:



What the crossover suggests: Six crossovers occur over the six-month window shown. Each time the faster 7-day MA (blue) crosses above the slower 30-day MA (red) — such as around mid-

February and late April — it signals that recent short-term momentum has turned upward relative to the longer trend, a classic "golden cross" buy signal. Conversely, when the 7-day MA crosses back below the 30-day MA — as seen around mid-March and late June — it signals fading short-term momentum, a "death cross" sell signal. Because this stock crosses back and forth fairly often, the firm should treat these as tactical, short-horizon signals rather than a single decisive long-term trend call.

10. Detrending Quarterly GDP (Python)

Task: An economist is analyzing a country's quarterly GDP data but needs to remove the long-term growth trend to study business-cycle fluctuations in isolation. Using Python, detrend the GDP series (e.g., by differencing or regression-based detrending), plot the detrended series, and interpret the remaining fluctuations.

Python Programming:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

np.random.seed(10)
n_quarters = 60
t = np.arange(n_quarters)
trend = 500 + 8*t
cycle = 25*np.sin(2*np.pi*t/16) + 10*np.sin(2*np.pi*t/6) # business-cycle
fluctuations
noise = np.random.normal(0, 5, n_quarters)
gdp = trend + cycle + noise

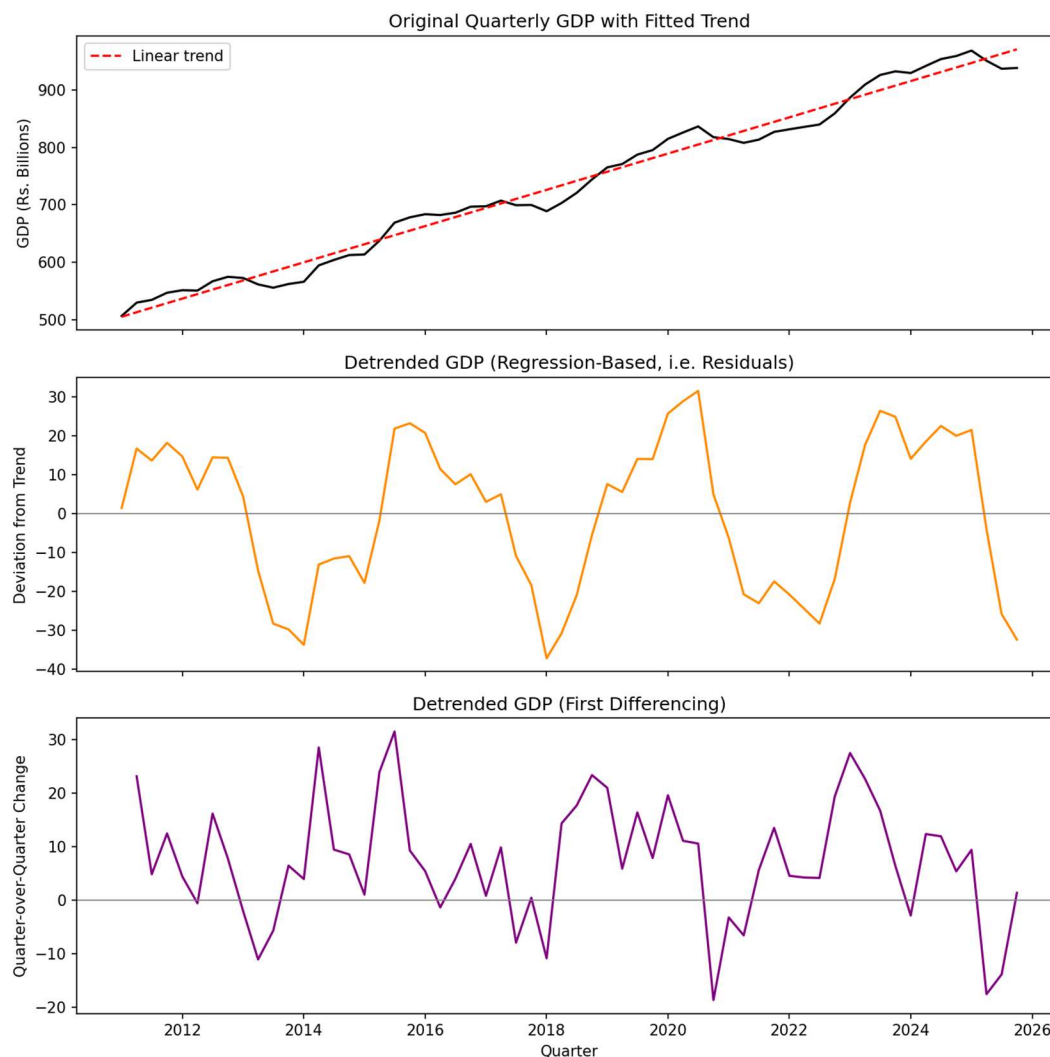
quarters = pd.period_range("2011Q1", periods=n_quarters, freq="Q")

# Method 1: regression-based detrending
X = np.vstack([np.ones(n_quarters), t]).T
coef, *_ = np.linalg.lstsq(X, gdp, rcond=None)
fitted_trend = X @ coef
detrended_regression = gdp - fitted_trend

# Method 2: first differencing
detrended_diff = pd.Series(gdp).diff()

fig, axes = plt.subplots(3, 1, figsize=(10,10), sharex=True)
axes[0].plot(quarters.to_timestamp(), gdp, color='black')
axes[0].plot(quarters.to_timestamp(), fitted_trend, color='red', linestyle='--',
            label="Linear trend")
axes[0].set_title("Original Quarterly GDP with Fitted Trend"); axes[0].legend()
axes[1].plot(quarters.to_timestamp(), detrended_regression, color='darkorange')
axes[1].axhline(0, color='grey', linewidth=0.8)
axes[1].set_title("Detrended GDP (Regression-Based, i.e. Residuals)")
axes[2].plot(quarters.to_timestamp(), detrended_diff, color='purple')
axes[2].axhline(0, color='grey', linewidth=0.8)
axes[2].set_title("Detrended GDP (First Differencing)")
axes[2].set_xlabel("Quarter")
plt.tight_layout()
plt.show()
```

OUTPUT:



Interpretation of remaining fluctuations: Once the steady long-term growth (about Rs. 7.9 billion of GDP added per quarter) is removed, both detrending methods reveal clear, repeating business-cycle swings that were invisible in the raw upward-sloping GDP line. The regression-residual view shows cycles of roughly ± 20 to ± 35 billion around the trend, alternating over multi-year spans, while the first-differenced view highlights the same cyclicity at a quarter-to-quarter resolution with somewhat more noise (its standard deviation is lower at ~ 11.2 vs ~ 19.1 for the residual method, since differencing only captures one-step changes rather than sustained deviation from trend). For the economist, this confirms that beneath the reassuring long-term growth trend, the economy is still going through genuine multi-year expansion-and-contraction cycles that a simple trend-only view would completely miss.